

【README】

作者：AI玩家日志

链接：<https://www.zhihu.com/question/1936375725931361485/answer/1982833664812413939>

来源：知乎

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

本文内容详细总结了大模型及Agent开发路线。

【1】先搞清楚一件事：AI Agent工程师到底要会什么？

我先给你泼盆冷水。

很多人（包括半年前的我）以为，AI Agent工程师就是"会调用LangChain API的人"。

错得离谱。

我后来总结了一下，这个岗位其实分三个层次：

第一层：API调用工程师（P5-P6，年薪30-50w）

1. 就是会用LangChain、LangGraph这些框架，能跑通官方demo，遇到问题就翻文档。
2. 说白了，就是个"高级调包侠"。
3. 这个层次的人，2025年已经烂大街了。
4. 我之前以为学到这个层次就够了，结果发现，这个层次的人面试通过率极低，因为供给

太多了。

第二层：系统开发工程师（P7-P8，年薪60-100w）

1. 能理解Agent的底层架构，知道ReAct、Plan-and-Execute这些模式是怎么回事，能设计复杂的多Agent协作系统，懂得在生产环境里优化性能。
2. 这是P7的门槛。
3. 大部分公司招AI Agent工程师，其实要的是这个层次的人。

第三层：基础设施架构师（P8+，年薪100w+）

1. 能从零实现一个Agent框架，深度理解LLM的推理机制，能设计大规模Agent集群的调度系统。
2. 这个层次的人，基本上是各大厂的专家、架构师了。
3. 我的目标是第二层，但我发现，想到第二层，你得有第三层的视野。
4. 不然面试官随便深挖几个问题，你就露馅了。

【2】说点实在的：到底要学什么？

我把这半年学的东西，按照"从底层到上层"的顺序，给你梳理一遍。

【2.1】向量数据库（这玩意儿比你想象的复杂）

1. 一开始我以为，向量数据库不就是"存Embedding，然后做相似度搜索"嘛，有啥难的？
2. 后来面试被问："为什么Pinecone用HNSW算法，Milvus支持多种索引？什么场景下该选哪种？"
3. 我才发现，我对向量数据库的理解，停留在"会用"的层面，根本不懂原理。
4. 后来我花了一个月时间，把向量检索的几个核心算法都搞明白了：
 1. **•HNSW**（分层图结构）：查询快，但内存占用大，适合高QPS的场景
 - IVF**（倒排索引+聚类）：适合大规模离线检索
 - Annoy**（随机投影树）：内存占用小，但召回

率稍低；

2. 然后我还自己动手，用Milvus搭了一个支持千万级向量的检索系统，踩了一堆坑：
3. •冷启动问题：新文档的Embedding怎么快速索引？
 1. •增量更新：怎么在不重建索引的情况下更新向量？
 2. •多租户隔离：怎么在共享集群里做租户级别的数据隔离？
4. 这些问题，B站教程里根本不会讲。但面试会问。

【2.2】RAG（别停留在Naive RAG）

我刚开始学RAG的时候，写的代码是这样的：

```
def naive_rag(query):  
    docs = vector_db.search(query, top_k=5)  
    context = "\n".join(docs)  
    response = llm.generate(f"Context: {context}\nQuery: {query}")  
    return response
```

我以为这就是RAG了。

1. 结果面试官问我："你这个RAG有什么问题？"
 1. 我：.....没问题啊？
 2. 面试官：检索质量差、上下文窗口浪费、无法处理多跳推理、缺乏可解释性。
 3. 我：.....（又是一片空白）
2. 后来我才知道，**Naive RAG**只是最基础的版本，生产环境里根本不够用。**真正的RAG**，要做这些优化：
 1. 第一步：Query优化：
 1. •Query Rewriting：把用户的问题改写成更适合检索的形式
 2. •Query Decomposition：把复杂问题拆成几个子问题
 3. •HyDE：先让LLM生成一个假设性的答案，再用这个答案去检索；
 2. 第二步：检索优化：

1. •Hybrid Search：向量检索+BM25，两个结果融合•Reranking：用Cross-Encoder重新排序
2. •Contextual Compression：把无关的内容压缩掉；
3. 第三步：生成优化：
 1. •Self-RAG：让模型自己判断要不要检索
 2. •CRAG：检测检索结果的质量，如果不行就回退到网络搜索

【小结】这些东西，我是自己一点点摸索出来的。B站教程里，基本不讲。

【2.3】Agent架构（这才是核心）

Agent这块，我踩的坑最多。一开始我以为，Agent就是"LLM + Tools"，让LLM调用几个工具就完事了。

后来我发现，**Agent的核心不是"调用工具"，而是"推理过程的设计"。

【2.3.1】ReAct模式（最基础但最重要）

1. ReAct就是让LLM交替进行"推理"和"行动"。

```
def react_agent(task):
    history = []
    while not is_finished():
        # 推理：下一步该做什么
        thought = llm.generate(f"Task: {task}\nHistory: {history}\nThought:")

        # 行动：执行工具
        action = parse_action(thought)
        observation = execute_tool(action)

        history.append({"thought": thought, "action": action, "observation": observation})

    return final_answer
```

2. 看起来简单吧？但实际上，这里面有一堆问题：

1. •推理错误怎么办？→ 需要Reflexion机制，让Agent反思自己的错误
2. •推理效率低怎么办？→ 需要Few-shot示例，提供高质量的推理样本
3. •任务太长怎么办？→ 需要分层ReAct，把任务拆成子任务

这些问题，我都是在实际项目里踩坑才知道的。

【2.3.2】Plan-and-Execute模式（适合复杂任务）

1. 这个模式是先让LLM生成一个完整的计划，然后逐步执行。

```
def plan_and_execute(task):
    # 生成计划
    plan = planner.generate_plan(task)

    # 执行计划
    results = []
    for step in plan:
```

```
result = executor.execute(step, context=results)
results.append(result)

# 如果执行失败，重新规划
if need_replan(result):
    plan = planner.replan(task, results)

return results
```

2. 这个模式的难点在于：

1. 怎么生成高质量的计划？ → 需要结构化输出，用JSON Schema约束
2. 什么时候触发重规划？ → 执行失败、发现新信息、用户需求变更
3. 哪些步骤可以并行？ → 需要分析步骤之间的依赖关系

【2.3.3】Multi-Agent协作（最复杂）

1. 这是我觉得最难的部分。

2. 怎么让多个Agent协作完成任务？我试过三种架构：

1. 中心化调度：一个主Agent负责分配任务给其他Agent；
2. 去中心化协商：Agent之间自己协商谁做什么
3. 分层管理：大Agent管小Agent

每种架构都有优缺点，具体用哪种，得看业务场景。

【2.4】Memory系统（这块容易被忽视）

1. 一开始我觉得，Memory不就是"把对话历史存起来"嘛。后来我发现，Memory系统的设计，直接影响Agent的智能程度。

2. 我把Memory分成三层：

1. 工作记忆，当前对话的上下文；

2. 短期记忆，定期总结（如每10条消息总结一次）；
3. 长期记忆，向量数据库；

第一层：工作记忆（就是当前对话的上下文）

```
class ConversationBuffer:
    def __init__(self, max_tokens=2000):
        self.messages = []

    def add_message(self, message):
        self.messages.append(message)
        # 超出token限制就删掉最早的消息
        while self.count_tokens() > self.max_tokens:
            self.messages.pop(0)
```

第二层：短期记忆（定期总结）

```
class SummaryMemory:
    def __init__(self):
        self.summary = ""
        self.recent_messages = []

    def add_message(self, message):
        self.recent_messages.append(message)

        # 每10条消息总结一次
        if len(self.recent_messages) > 10:
            self.summary = llm.summarize(self.summary,
self.recent_messages)
            self.recent_messages = []
```

第三层：长期记忆（向量数据库）

```
class VectorMemory:
    def store(self, memory_item):
        self.vector_db.insert({
            "text": memory_item.text,
            "embedding": embed(memory_item.text),
            "timestamp": memory_item.timestamp,
            "importance": memory_item.importance
        })

    def retrieve(self, query):
        return self.vector_db.search(query, top_k=5)
```

这套Memory系统，我是参考人类的记忆机制设计的。

效果还不错，但实现起来挺麻烦的。

【2.5】生产化工程（这是P7+的分水岭）

前面那些都是“能跑”的层面，但生产环境还要考虑：可观测性（llm执行链路追踪），成本优化，安全性；

【2.5.1】可观测性（怎么debug一个失败的Agent？）

传统后端系统，你可以看日志、看Trace。

但Agent系统，一个任务可能涉及几十次LLM调用，每次调用的输入输出都不一样，怎么追踪？

我自己实现了一个简单的追踪系统：

```
class AgentTracer:
    def start_span(self, name, inputs):
        span = {
            "span_id": generate_id(),
            "name": name,
```



```
        "start_time": time.time(),
        "inputs": inputs
    }
    self.spans.append(span)
    return span

def end_span(self, span_id, outputs):
    span = self.find_span(span_id)
    span["end_time"] = time.time()
    span["outputs"] = outputs
    span["duration"] = span["end_time"] - span["start_time"]
```

有了这个，我就能看到Agent的完整推理链路，哪一步出问题了一目了然。

【2.5.2】成本优化（怎么省钱？）

1. LLM调用是要花钱的，而且不便宜。我总结了几个省钱的技巧：
 1. **智能模型路由**：简单任务用便宜的模型，复杂任务用贵的模型
 2. **Prompt压缩**：用LLMLingua这种工具，把Prompt从500 tokens压缩到200 tokens
 3. **语义缓存**：相似的问题直接返回缓存的答案

这些优化做完，成本能降低30-50%。

【2.5.3】安全性（怎么防止Agent被攻击？）

1. 这块我一开始完全没意识到，后来看到有人用Prompt Injection攻击Agent，我才知道这事儿有多严重。
2. 主要防御三个方面：
 1. **输入验证**：检测用户输入里有没有注入攻击
 2. **工具访问控制**：限制Agent能调用哪些工具
 3. **输出验证**：检查Agent的输出有没有泄露敏感信息

【3】学习路径：我是怎么从0到1的

说了这么多理论，你肯定想知道，**具体怎么学**？我把我这半年的学习路径，按照时间线给你梳理一遍。

【3.1】第1-2个月：打基础

1. Week 1-2: LLM基础

1. •我先把《Attention Is All You Need》这篇论文啃了一遍（说实话，第一遍看得云里雾里）
2. •然后自己动手，用PyTorch实现了一个简单的Transformer
3. •这个过程很痛苦，但确实让我理解了LLM的底层原理

2. Week 3-4: Prompt Engineering

1. •学习Few-shot、Chain-of-Thought这些技巧；
2. •自己设计了一个Prompt模板库，积累了一些好用的Prompt；

3. Week 5-8: RAG实践

1. •搭了一个完整的RAG系统，从文档上传到向量化到问答；
2. •对比了不同的Embedding模型（OpenAI、Cohere、BGE）；
3. •实现了Hybrid Search + Reranking；

4. Week 9-12: 向量数据库

1. •深度使用Milvus，把官方文档翻了个遍；
 2. •理解HNSW、IVF这些算法的原理；
 3. •自己搭了一个千万级向量的检索系统，踩了无数坑；
-

【3.2】第3-4个月：深入Agent

1. Week 13-16: Agent基础

1. •精读ReAct、Reflexion这些论文
2. •从零实现了一个ReAct Agent (没用任何框架, 全手写)
3. •这个过程让我真正理解了Agent的状态管理

2. Week 17-20: LangGraph深度

1. •学习StateGraph的设计模式
2. •实现了复杂的Agent工作流, 包括条件分支、循环、并行执行
3. •构建了一个Plan-and-Execute Agent

3. Week 21-24: Multi-Agent系统

1. •设计Agent通信协议
2. •实现Agent编排系统
3. •处理冲突和容错

【3.3】第5-6个月：生产化

1. Week 25-28: 可观测性

1. •设计Agent追踪系统
2. •实现指标收集和监控
3. •构建可视化Dashboard

2. Week 29-32: 性能优化

1. •LLM调用优化 (缓存、批处理) ;
2. •成本控制策略;
3. •并发和异步处理;

3. Week 33-36: 安全与可靠性

1. •实现输入输出验证
 2. •工具访问控制
 3. •错误处理和重试机制
-

【4】 面试经验： P7级别到底考什么？

我前前后后面了5家公司，有过的有挂的，总结了一些经验。

【4.1】 考点1： 系统设计题（必考）

1. 典型问题： "设计一个能够自动处理客户工单的Agent系统"

1. 这种题，考的是你的架构设计能力。
2. 我的回答框架是这样的：
 1. 先问清楚需求**（千万别上来就开始设计）
 1. •工单类型有哪些？
 2. •并发量多大？
 3. •准确率要求多高？
 4. •延迟要求多少？
 2. 画架构图：
 1. •整体架构
 2. •核心模块
 3. •数据流
 3. 深入细节：
 1. •Agent怎么设计？

- 2. •工具怎么设计?
- 3. •状态怎么管理?
- 4. •错误怎么处理?

4. 优化方案:

- 1. •性能怎么优化?
- 2. •成本怎么控制?
- 3. •怎么扩展?

【4.2】考点2：算法与原理（区分度很高）

1. 典型问题: "解释HNSW算法的原理，以及为什么它比暴力搜索快"

- 1. 这种题，考的是你对底层原理的理解。
- 2. 如果你只是会用，肯定答不上来。

【4.3】考点3：实战经验（最重要）

1. 典型问题: "你遇到过Agent陷入无限循环的情况吗？怎么解决的？"

- 1. 这种题，考的是你有没有真正做过项目。

2. 我的回答是:

- 1. "遇到过。有一次Agent在处理一个复杂任务时，一直在'推理-行动-推理-行动'这个循环里出不来。后来我分析了一下，发现是因为Agent的推理结果不够明确，导致它一直在尝试不同的工具，但都没有得到满意的结果。
- 2. 我的解决方案是：
 - 1. 设置最大循环次数，超过就强制退出；
 - 2. 在每次循环时，让Agent判断'是否取得了进展'；如果连续3次没进展就退出
 - 3. 优化Prompt，让Agent的推理结果更明确

这个问题解决之后，Agent的成功率从60%提升到了85%。这种回答，面试官一听就知道你是真做过的。

【5】资源推荐：别走弯路

最后，我把这半年用过的、觉得真正有用的资源，分享给你。

【5.1】必读论文（按重要性排序）

1. ReAct: Synergizing Reasoning and Acting in Language Models**
 1. •这是Agent的基础
2. Reflexion: Language Agents with Verbal Reinforcement Learning
 1. •讲Agent怎么从错误中学习
3. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**
 1. •RAG的奠基论文

【5.2】实战项目（从简单到复杂）

【5.2.1】智能文档问答系统

1. •技术栈：LangChain + Milvus + GPT-4
2. •学习重点：RAG pipeline设计

【5.2.2】自动化代码审查Agent

1. •技术栈：LangGraph + GitHub API + GPT-4
2. •学习重点：Tool使用、结构化输出

【5.2.3】Multi-Agent协作系统

1. •技术栈：LangGraph + Custom Tools
 2. •学习重点：Agent编排、通信协议
-

【5.3】信息源（保持技术敏感度）

1. •arXiv：每周看cs.AI和cs.CL的最新论文
 2. •GitHub Trending：关注AI Agent相关的热门项目
 3. •Twitter/X：关注AI领域的KOL
 4. •Discord/Slack：加入AI开发者社区
-

【6】写在最后：别被焦虑裹挟

1. 说实话，这半年我也很焦虑。
2. 看着身边的人一个个转型成功，我还在自己摸索，压力真的很大。
 1. 但现在回头看，这半年的积累，是值得的。
 2. 我不仅学会了AI Agent的技术，更重要的是，我建立了一套系统化的学习方法。
 3. 这套方法，比技术本身更值钱。 **
3. 最后说一句：AI这个领域变化太快，没有人能一直领先。但只要你保持学习，保持思考，你就不会被淘汰。加油吧，兄弟。